CS683 Project Assignment

iperf3 network tester

Jerold (Jerry) Abramson

1. Overview

The application being developed will utilize the de-facto network protocol tool, `iperf3`, for testing network speeds between a client and a server.

1.1. More Details

- For my project I am implementing a network performance monitor based on the iPerf3 protocol.
- Although there are a couple of Android apps that already partially implement this, one of them is terribly complex, quite expensive, (Analti), more focused on Wi-Fi and internet performance monitoring.
- There is also a very old version that is nothing more than the text-based version of the existing 'iperf3' application that runs in a console terminal window.
- There is a nice version that runs on iOS, but that has not been ported to Android.

1.2. Functional Requirements and limitations

- The application **will only be implementing** the client-side functionality of iPerf3:
  - It will assumed that an iPerf3 server is running on another host.
    - I will be providing a mechanism to ensure that a server will be available to the instruction team during analysis and grading.
- The actual `iperf3` protocol **will not be** reimplemented.
  - The official downloads are available here:

ABI	Binaries
arm64-v8a	here
armeabi-v7a	here
x86	here
x86_64	here

1.3. Challenges resoved

- I have solved working through the mechanism to ensure that the `iperf3` binary is uploaded to the Android device/emulator.
- I have also solved having the `iperf3` binary being executable inside the standard Android sandbox security architecture.
- I am usingng the standard Kotlin/Java ProcessBuilder class APIs.
- I have a script that can be run by the testing team to allow for the iperf3 exectuable to run.
  - The basics of the workaround are to disable SELinux on the emulator, which allows for the binaries included with the application to be e3xecutable.

1.4. Future Goals

- I have studied a GitHub project that incorporates the iperf3 binary into a full Android NDK application.
- I even moved on further and started converting my application to utilize this approach.
  - The native `iperf3` native libraries (`.so`) **will not be** utilized using direct JNI APIs from Kotlin/Android.
  - The official releases of the `iperf3` binary for Android (all hardware architectures and emulators) will be utilized for this project.
  - I have put that effort into another branch:
    - https://github.com/CS683/project-jerryabramson/tree/tryembeddediperf3
- For the course project I will continue with the current approach of using an external iperf3 binary.

1.5. Current challenges to meet the functional requirements

The code currently is hard-coding to the dark theme, as I could not get the correct Material Theme composition working using the Kotlin code in UI.

2. Motivation

I utilize network testing tools as part of my home-labb'ing hobby.
One of the tools I am constantly utilizing is based on the iPerf3 protocol, which is defined here:
The full iperf3 source code and technical documentation for iperf3 is available on this website:

- *iPerf - The TCP, UDP and SCTP network bandwidth measurement tool*

The full source code and technical documentation for the iperf3 implementation on Android is available here:

- GitHub - davidBar-On/android-iperf3: Pre-compiled iperf3 binaries for Android + Dockerfile with SDK and NDK for manual build · GitHub

I utilize network testing tools as part of my "home-labb'ing" hobby.

- One of the tools I am constantly utilizing is based on the iPerf3 protocol, which is defined here:
- *For some context on iPerf3 is available* → here

3. Purpose

Provide the ability to test WiFi signal strength using an Android phone.

3.1. Potential User

- `iperf3` is a well-known tool for networking professionals.
- My goal is to launch this application on the Android Play Store over the summer for a one-time purchase price of $0.99
-

3.2. Related Work

I have developed a Java text-based console program that runs the `iperf3` native executable in the background and provides performance details in an asynchronous manner.

The application supports running at the command-line with ‘fancy’ ANSI line drawing and fonts, as well as standard text-based output suitable for scripts.
Here are some examples of this application running in standard text-based mode

- *Note: pve is a local hostname reachable in my LAN.*

3.2.1. Text-Based Mode

```
TERM=dumb LANG=C newTestBandWidth.sh pve -R
[Sat Mar 28 13:10:52 EDT 2026] Executing Command-line: '/opt/local/bin/iperf3' '--
forceflush' '--connect-timeout' '3000' '-c' 'pve' '-O 2' '-P 8' '-t' '10' '-R'
Initiating [ ..........Connecting to host pve, port 5201
Reverse mode, remote host pve is sending
  Local Host/IP: 192.168.1.10 remote Host/IP: 192.168.1.7 Remote Port: 5201
....XXXXX   Skipping 0.00-1.01          2.35  Gbits/sec  (omitted)
....XXXXX   Skipping 1.01-2.00          2.35  Gbits/sec  (omitted)
....XXXXX   Running  0.00-1.01          2.35  Gbits/sec
....XXXX   Running  1.01-2.01          2.35  Gbits/sec
....XXXXX   Running  2.01-3.00          2.35  Gbits/sec
....XXXXX   Running  3.00-4.01          2.35  Gbits/sec
....XXXXX   Running  4.01-5.01          2.35  Gbits/sec
....XXXXX   Running  5.01-6.00          2.35  Gbits/sec
....XXXXX   Running  6.00-7.01          2.35  Gbits/sec
....XXXXX   Running  7.01-8.00          2.35  Gbits/sec
....XXXXX   Running  8.00-9.00          2.35  Gbits/sec
   Running  9.00-10.01         2.35  Gbits/sec
Results:   0.00-10.01         2.35  Gbits/sec  sender
        => 0.00-10.01         2.35  Gbits/sec  receiver

[Avg]     2.35  Gbits/sec
[Min]     2.35  Gbits/sec
[Max]     2.35  Gbits/sec
  ------------------------------------------------------------------------------
  Return Code: 000 [Avg=2.35 Gbits/sec]
```

3.2.2. “Fancy” line drawing and fonts

```
🚀 newTestBandWidth fedoranas -R
[Sat Mar 28 13:16:38 EDT 2026] Executing Command-line: 'C:/Program Files/iperf3.12_64/iperf3.exe' '--forceflush' '--conn
ect-timeout' '3000' '-c' 'fedoranas' '-O 2' '-P 8' '-t' '10' '-R'
Connecting to host fedoranas, port 5201
Reverse mode, remote host fedoranas is sending
  Local Host/IP: 192.168.1.20 remote Host/IP: 192.168.1.204 Remote Port: 5201
      Running  1.00-2.00   - (          ]      7.61  Gbits/sec
```

```
🚀 newTestBandWidth fedoranas -R
[Sat Mar 28 13:16:38 EDT 2026] Executing Command-line: 'C:/Program Files/iperf3.12_64/iperf3.exe' '--forceflush' '--conn
ect-timeout' '3000' '-c' 'fedoranas' '-O 2' '-P 8' '-t' '10' '-R'
Connecting to host fedoranas, port 5201
Reverse mode, remote host fedoranas is sending
  Local Host/IP: 192.168.1.20 remote Host/IP: 192.168.1.204 Remote Port: 5201
  Results:   0.00-10.00          ]      7.31  Gbits/sec  sender
          => 0.00-10.00                 7.31  Gbits/sec  receiver
                            [Avg]       7.31  Gbits/sec
                            [Min]       5.59  Gbits/sec
                            [Max]       9.23  Gbits/sec

  Return Code: 000 [Avg=7.31 Gbits/sec]
```

3.2.3. Similar Android Applications

There are a few similar implementations of this tool on the Android play store, shown in the below table:

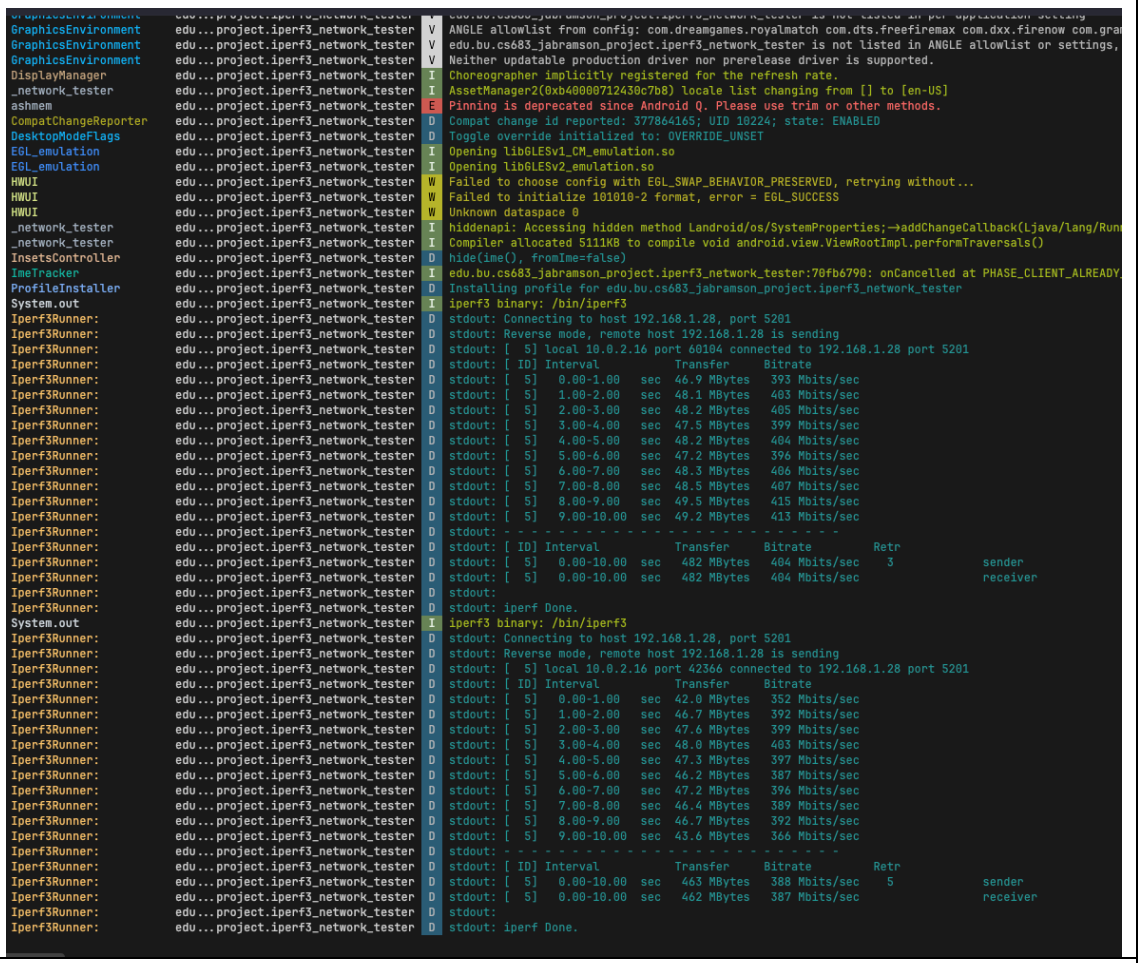
App Name	Play Store Link
Analti | *analiti Networking Experts*	analiti - Speed & WiFi - Apps on Google Play
iperf3 | *Uncle Tools*	iperf3 - Android Apps on Google Play

3.2.4. Similar iOS application

App Name	App Store Link
iPerf3 WiFi Speed Test	iPerf 3 Wifi Speed Test App - App Store

4. Requirement Analysis and Testing

4.1. Requirement 1 - DONE

Title	**prototype of remote IP connection (ESSENTIAL)**
Description	A hard-coded remote IP address will be used as the remote iPerf3 server Hard-coded command-line arguments will be used for iPerf3
Acceptance Tests	runs a fixed loop of 10 iperf3 intervals over TCP/IP in the background
Test Results	Initially the output may only be visible via logcat I managed to get the output to display after the test is completed. I still need to work on the asyncronous behavior so we can see the speed results as they are presented (every second).
Mockup	
Status	Iteration 1 - Completed Iteration 2 (or possibly 3) – Update the Composable URI

4.2. Requirement 2 – DONE – User Interface largely implemented, see screen

Title	**User Interface Displays Interaction and Status**
Description	After the user presses the button 'Run Iperf-3 Test' the execution runs in the background, with the user interface providing a progress bar.
Mockups	
Acceptance Tests	The progress bar moves forward while the background job is running.
Test Results	The progress bar moves forward, and the 'Run Iperf-3 Test' button is greyed out. Once the background job is completed, the progress bar shows 'Complete' and the button is again usable (no longer greyed out)
Status	Iteration 1 – Job runs in background Iteration 2 – As the job is running in the background, the progress bar is updated with the most recent bandwidth result i.e [=====] Interval 5: 393 Mbits/sec Iteration 3 – Accurate results based on the optional inputs (i.e. hostname, run count, parallel streams, etc…)

4.3. Requirement 3 - DONE

Title	**Server Address and other options**
Description	The user shall be able to type in a valid server address of a remote iperf3 server
Mockups	
Acceptance Tests	host-name/IP Address validation
Test Results	Positive Test: "localhost" **Test runs** Negative Test: "abcd.baddomain" *Validation error*
Status	Iteration 1 - Allow entry of server name Iteration 2 - Validates remote host before execution Iteration 3 - perform iperf3 test with hard-coded-count of 10 against provided host Iteration 4 – All command-line arguments supported: • Server Address • Server Port • Transmit Mode (Upload/Download) • Parallel Streams • Skip Iteration Count • Test Duration • Output Units • Loop count • Share as…button saves output somehow

4.4. Requirement 4 – In Progress

Title	**Unit of measurement**
Description	The user shall be able to specify the unit of measurement for output bandwidths
Mockups	Entry of MB, Entry of gb, Entry of bytes
Acceptance Tests	Validate rationale unit text
Test Results	Positive: MB, Negative foobar
Status	Iteration 1 - Allow keyboard entry of unit Iteration 2 - validate unit of measurement Normalize output results to specified unit

5.

5. Design and Implementation

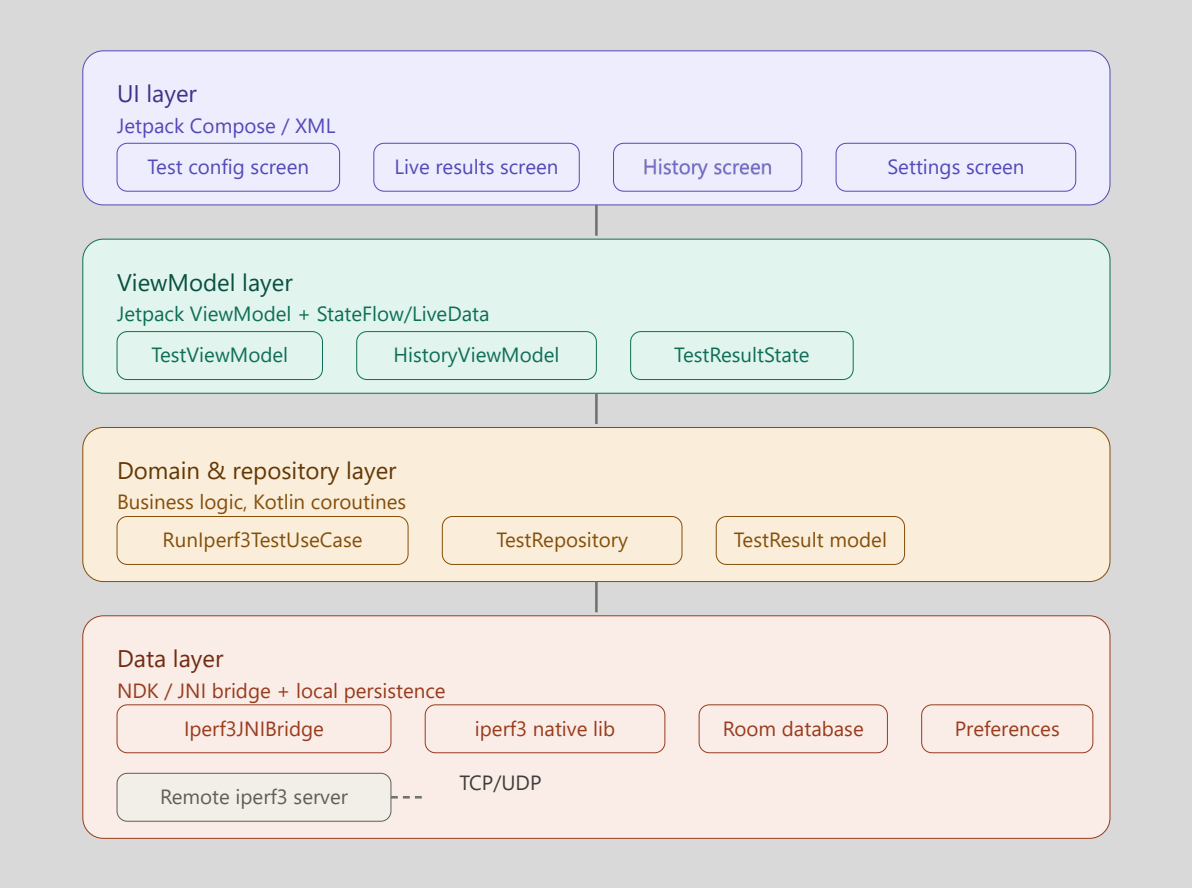
I utilized the new https://claude.ai/ capability for render of diagrams without the need for Canvas, PlantUML, or other tools.

AI Prompt:

> Design the basic architecture for an Android application that performs iperf3 tests against a remote iperf3 server

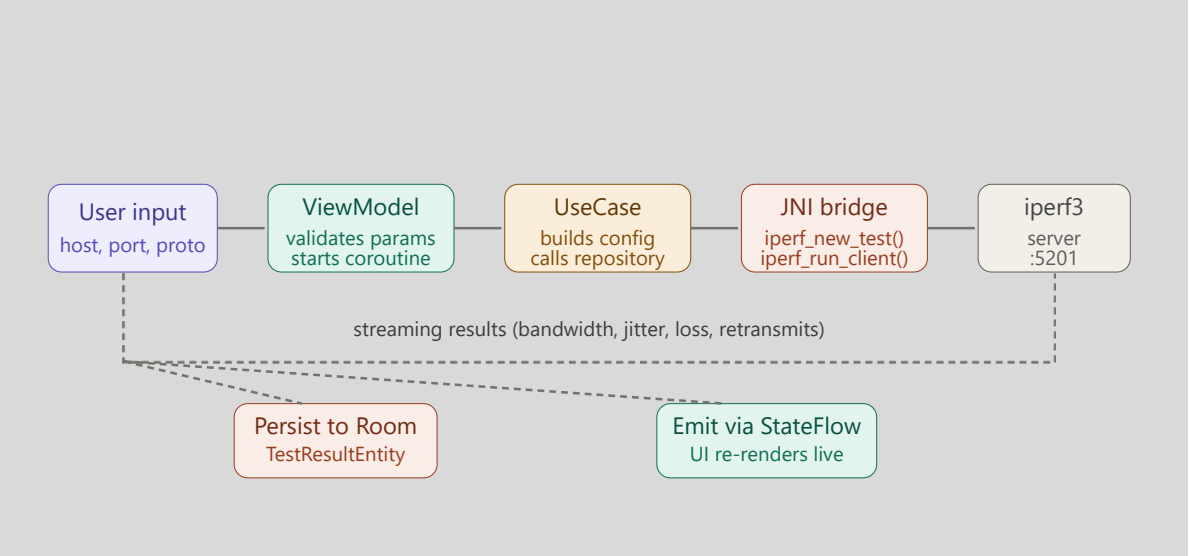
5.1. Layered Architecture

- The app is organized into four tiers from UI down to the native iperf3 binary.
- Unlike the AI provided Architecture, **I will not be utilizing an NDK/JNI Bridge** but instead will be executing the iperf3 binary and parsing its output.
- I will be putting together a full set of new diagrams for the final project submission.



5.2. Test Execution Flow

Instead of utilizing the JNI bridge, I will be using the standard Kotlin/Java ProcessBuilder APIs to run the iperf3 binary in the background.
I will be putting together a full set of new diagrams for the final project submission.



5.3. Full component breakdown organized by prose

Here's a breakdown of the architecture, organized across three views: the layered app architecture, the data flow for a test run, and the component detail.

I will be rewriting this section for the final submission.

Layered architecture — the app is organized into four tiers from UI down to the native iperf3 binary:**Test execution flow** — what happens when the user taps "Run test":---

Here's the full component breakdown in prose, organized by layer:

UI layer (Jetpack Compose) — four screens: a test configuration screen (host, port, protocol, duration, parallel streams, direction), a live results screen with a real-time chart of throughput, a history screen backed by a RecyclerView or LazyColumn, and a settings screen for saved server profiles.

ViewModel layer — `TestViewModel` owns the coroutine scope and exposes a `StateFlow<TestUiState>` that the UI collects. States cycle through `Idle → Connecting → Running(progress) → Complete(result) → Error(reason)`. `HistoryViewModel` paginates results from Room via a Paging 3 data source.

Domain layer — `RunIperf3TestUseCase` accepts a `TestConfig` data class (host, port, protocol, duration, parallel, reverse) and returns a `Flow<TestEvent>` so intermediate interval results stream up while the test runs. A `TestRepository` interface abstracts both the JNI bridge and the local database so the domain layer stays decoupled from Android specifics.

Data layer — the critical piece — iperf3 is a C library, so it's compiled as a shared object (`.so`) via the Android NDK and bundled in `src/main/jniLibs/`. A thin JNI wrapper (`Iperf3Bridge.kt` + `iperf3_bridge.cpp`) translates Kotlin calls into `iperf_new_test()`, `iperf_set_*()`, and `iperf_run_client()` calls from `libiperf.h`. Interval callbacks from the C library are marshalled back to Kotlin via a JNI callback interface and posted onto a `Channel<TestEvent>`. Room stores `TestResultEntity` rows with columns for timestamp, host, protocol, bandwidth (bps), jitter (ms), packet loss (%), and retransmits. A DataStore `Preferences` instance persists default server profiles and app settings.

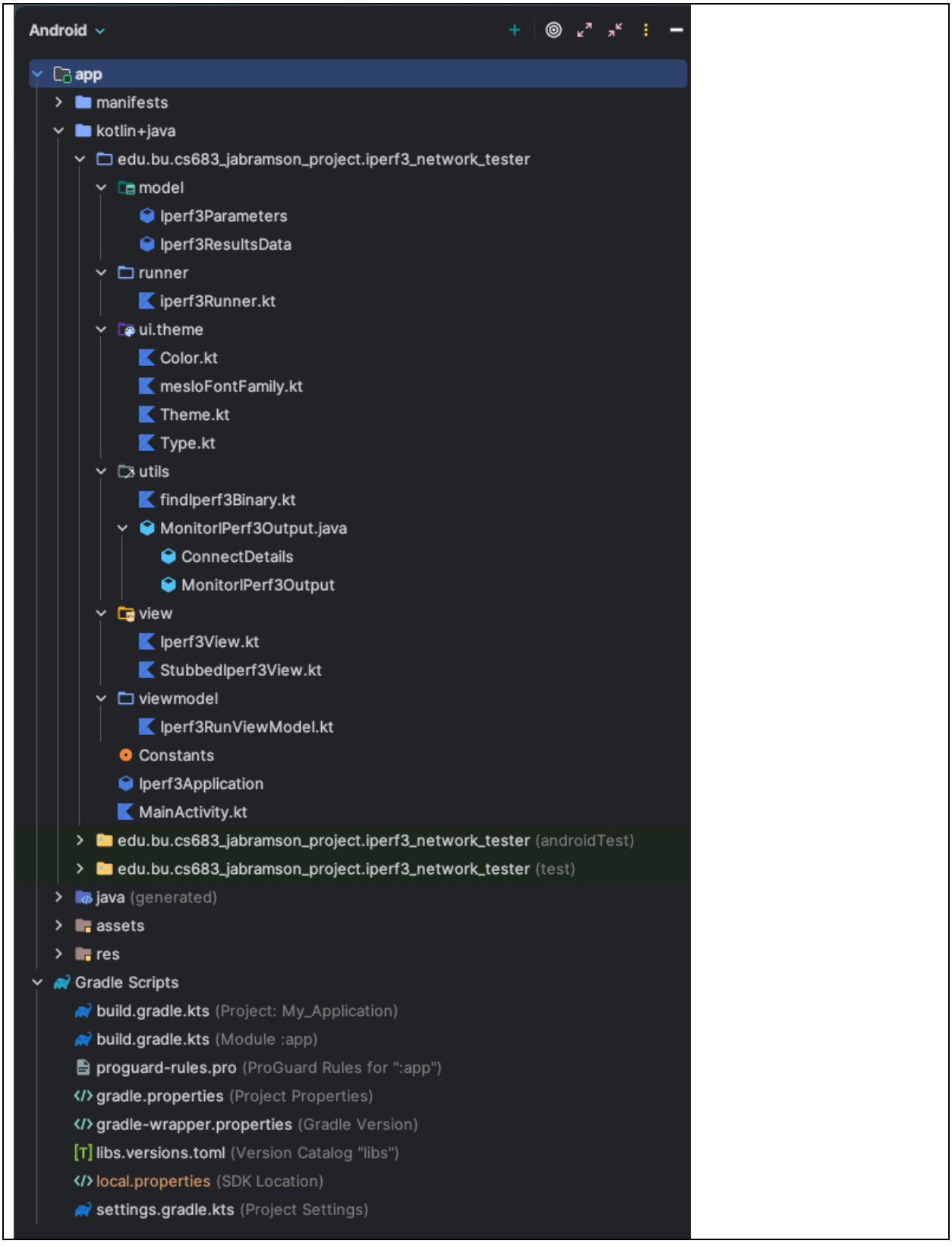
Key design decisions to make early:

The biggest one is whether to use `libiperf` directly via JNI (cleaner, streaming results, no process overhead) or shell out to the `iperf3` binary via `ProcessBuilder` (much easier to implement but requires bundling the binary and dealing with stdout parsing). The JNI approach is the right one for a production app.

For networking, `iperf3` opens its own socket to port 5201 (configurable) on the server — the app just needs `INTERNET`permission. You'll also want `ACCESS_NETWORK_STATE` to gate tests on connectivity and to tag sockets with `TrafficStats`for per-UID accounting.

Concurrency should run entirely on `Dispatchers.IO` inside a `viewModelScope` coroutine, with the native blocking `iperf_run_client()` call wrapped in `withContext(Dispatchers.IO)` to avoid blocking the main thread. Cancel support comes from a `Job` that, when cancelled, calls `iperf_reset()` on the native side.

2. Project Structure



6. Timeline

Iteration	Application Requirements • Essential • Desirable • Optional	Android Components and Features to be used
0.5	Redone to utilize Jetpack Compose	Jetpack Compose
1 DONE and thrown away	Essential: Stubbed version of progress bar	TBD (Networking)
COMPLETED 1	Essential: Application executes in background against a real host using the iperf3 executable	TextView
2 COMPLETED	As the iperf3 test is running in the background, provide a running total of the current bitrate	
2	After the test completes, provide a Min, Max, and Avg bitrate	
3 COMPLETED Test harness is in development	Ensure all possible network issues are handled gracefully	
2	Essential: Manual keyboard entry of test count	TextView
1		
2 or 3 COMPLETED EARLY Iteration 1	Essential: perform a single iperf3 test (count=1)	TBD (Consuming iperf3 native executable)
2 May or may not need this??	Desirable: Validate remote host	Networking
3 – DONE EARLY	Essential: Perform iperf3 to specified host with a count of 1	TBD
4 – DONE EARLY	Essential: iperf3 runs against specified host for specified count with specified unit	TBD

7. Current Status

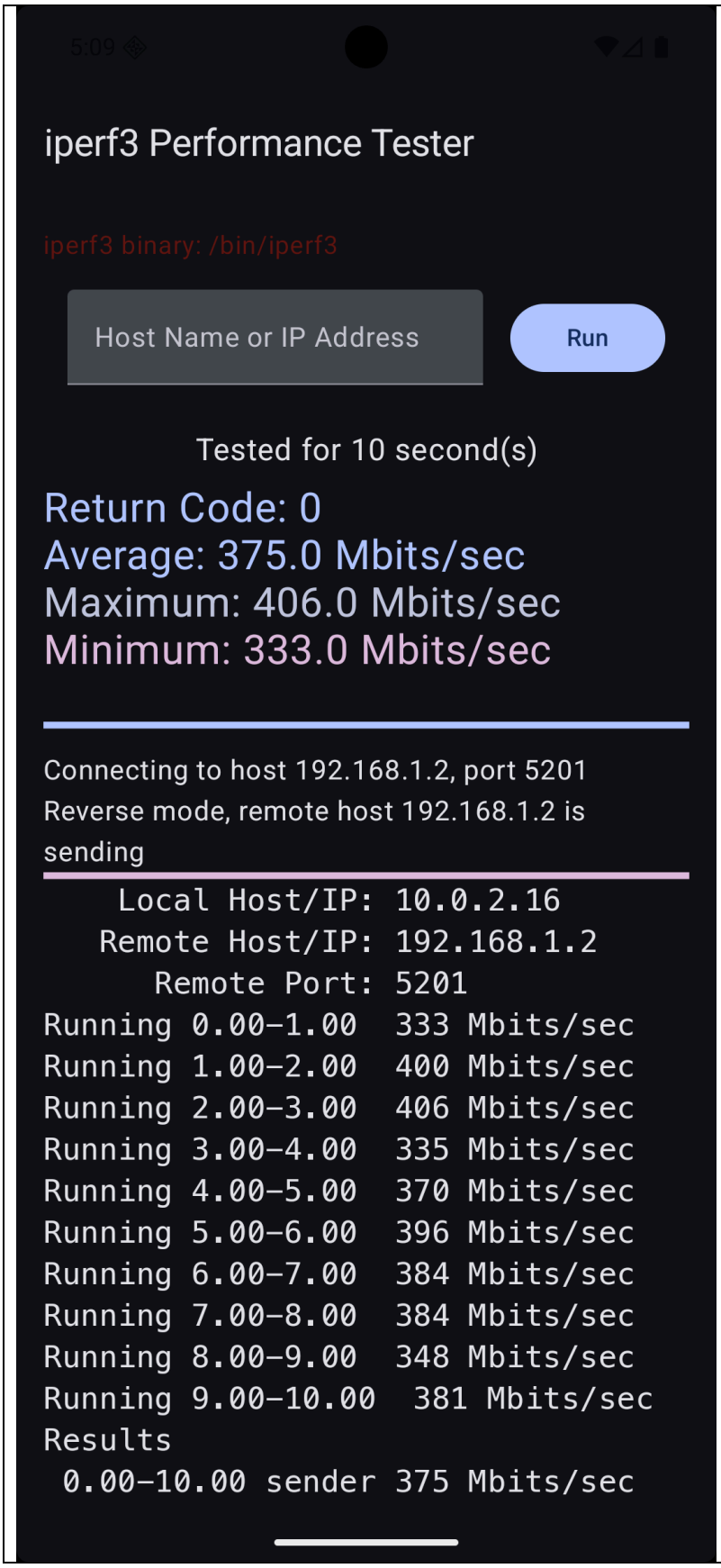
Screen shots of current status are below

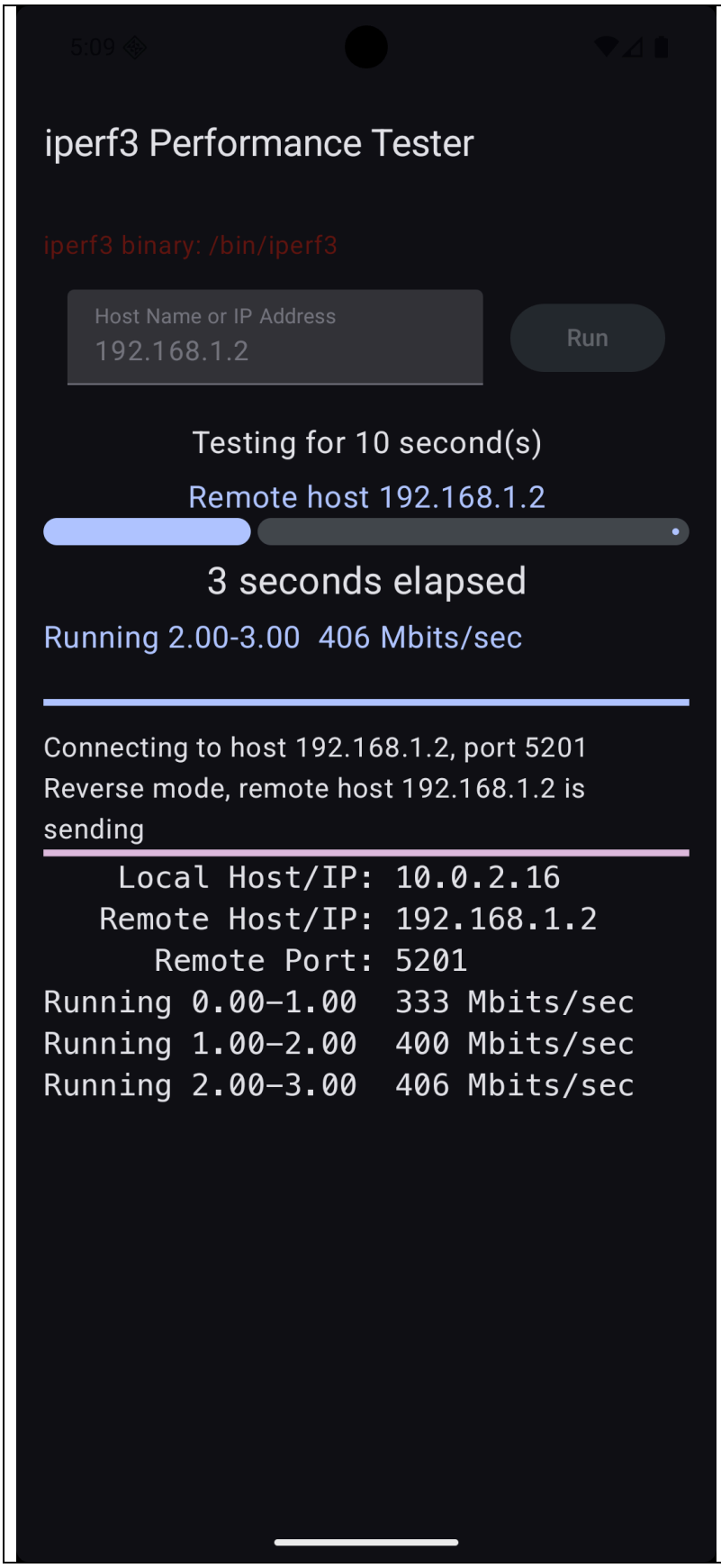
- For the best experience use one of the simulators below.
  - I have built in a simulation in case you have issues.
  - I have also put together a small script that is executed prior to build.
    - This script changes the Android permissions to disable SELinux.
    - This will only work for certain simulators (include those shown below).
    - It will also work for root'ed Android devices.

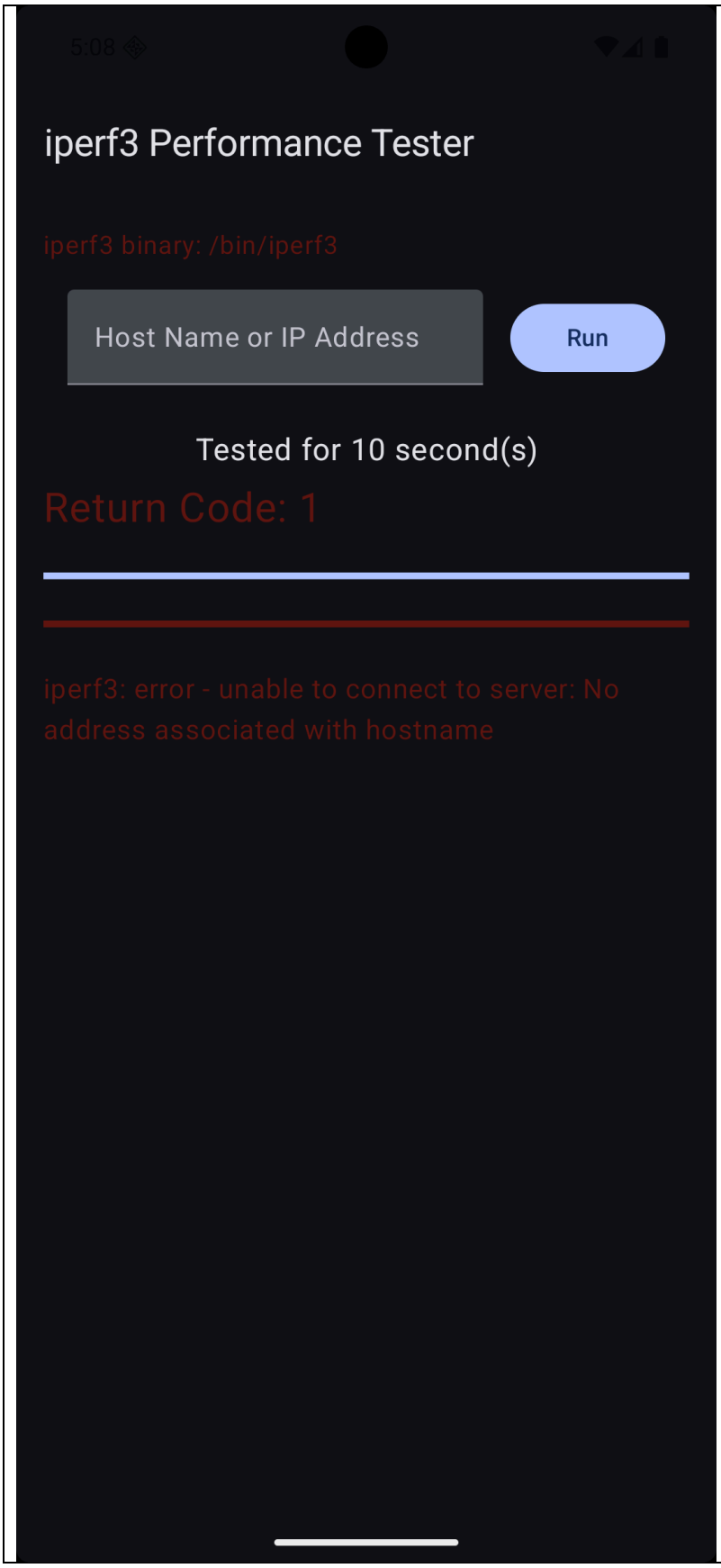
---

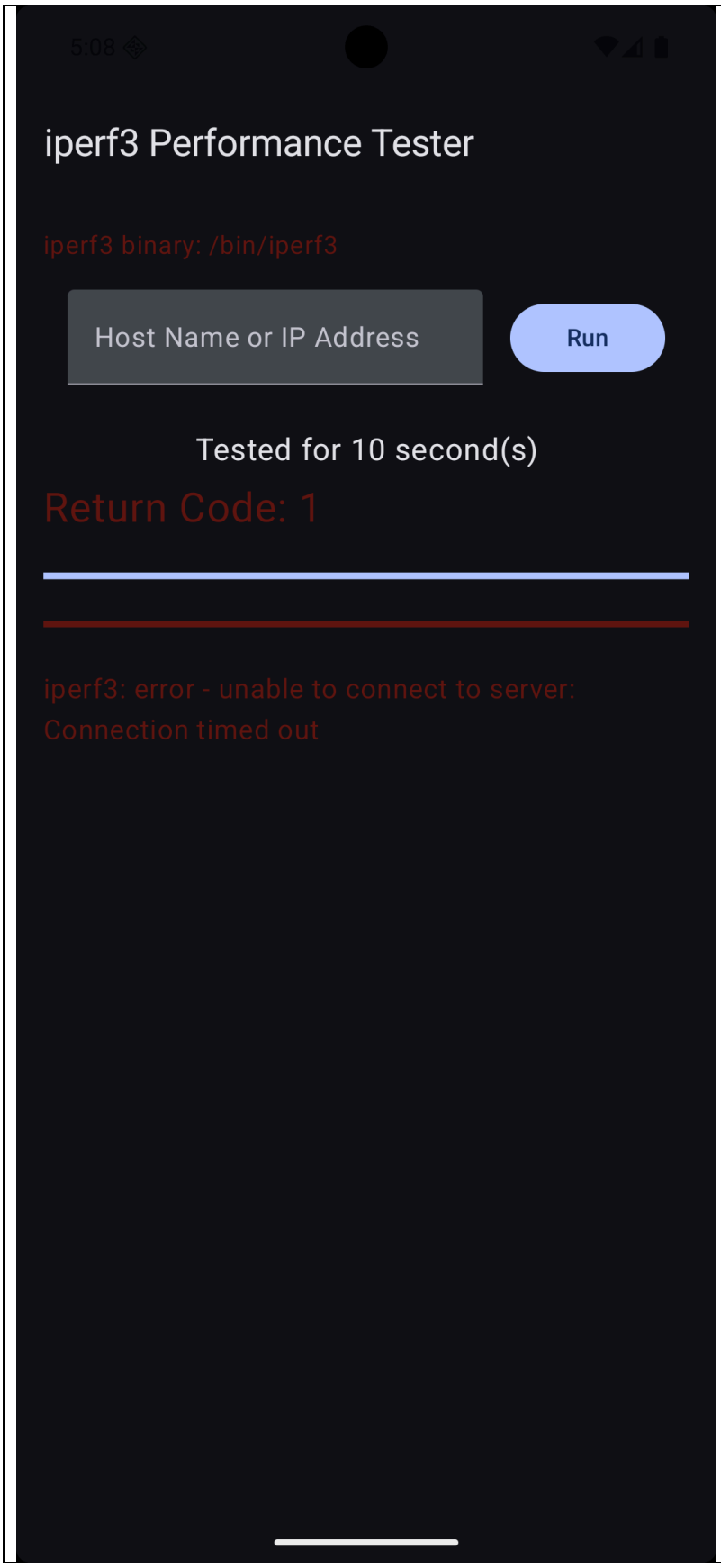
macOS: :Pixel 6 Android 16.0 ("Baklava") | arm64

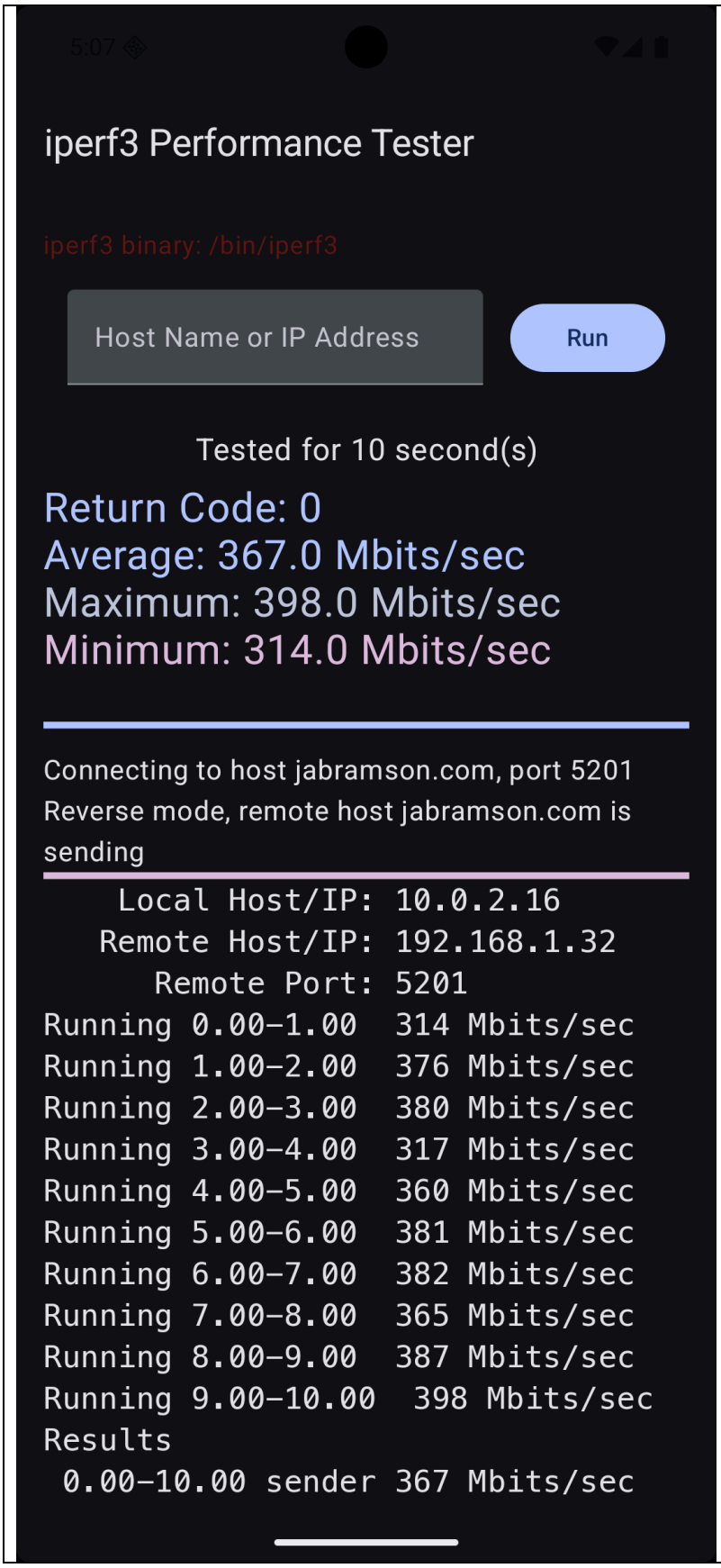
Windows: Pixel 6 Android 16.0 ("Baklava") | x86_64

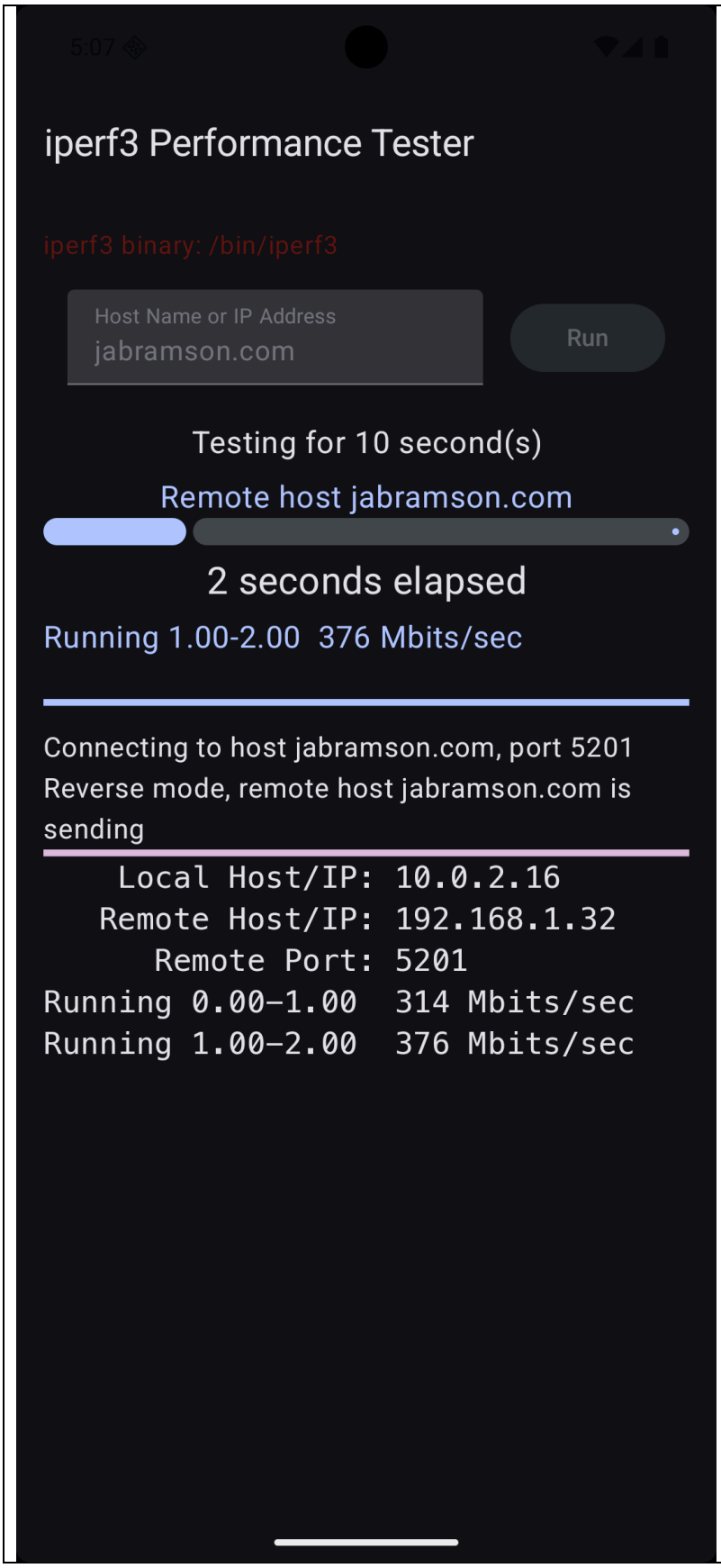

5:09
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
Run
Tested for 10 second(s)
Return Code: 0
Average: 375.0 Mbits/sec
Maximum: 406.0 Mbits/sec
Minimum: 333.0 Mbits/sec
Connecting to host 192.168.1.2, port 5201
Reverse mode, remote host 192.168.1.2 is sending
Local Host/IP: 10.0.2.16
Remote Host/IP: 192.168.1.2
Remote Port: 5201
Running 0.00-1.00 333 Mbits/sec
Running 1.00-2.00 400 Mbits/sec
Running 2.00-3.00 406 Mbits/sec
Running 3.00-4.00 335 Mbits/sec
Running 4.00-5.00 370 Mbits/sec
Running 5.00-6.00 396 Mbits/sec
Running 6.00-7.00 384 Mbits/sec
Running 7.00-8.00 384 Mbits/sec
Running 8.00-9.00 348 Mbits/sec
Running 9.00-10.00 381 Mbits/sec
Results
0.00-10.00 sender 375 Mbits/sec

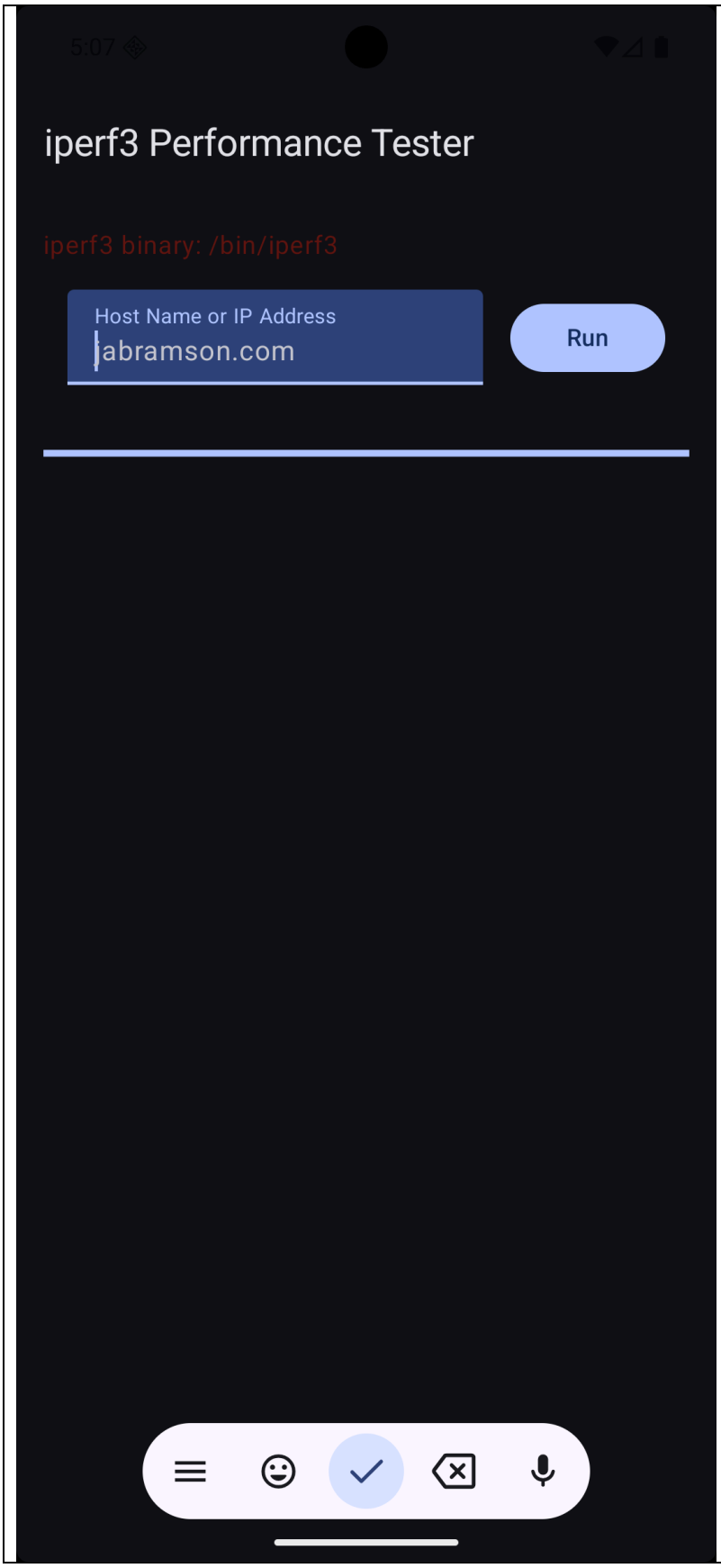

5:09
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
192.168.1.2
Run
Testing for 10 second(s)
Remote host 192.168.1.2
3 seconds elapsed
Running 2.00-3.00 406 Mbits/sec
Connecting to host 192.168.1.2, port 5201
Reverse mode, remote host 192.168.1.2 is sending
Local Host/IP: 10.0.2.16
Remote Host/IP: 192.168.1.2
Remote Port: 5201
Running 0.00-1.00 333 Mbits/sec
Running 1.00-2.00 400 Mbits/sec
Running 2.00-3.00 406 Mbits/sec

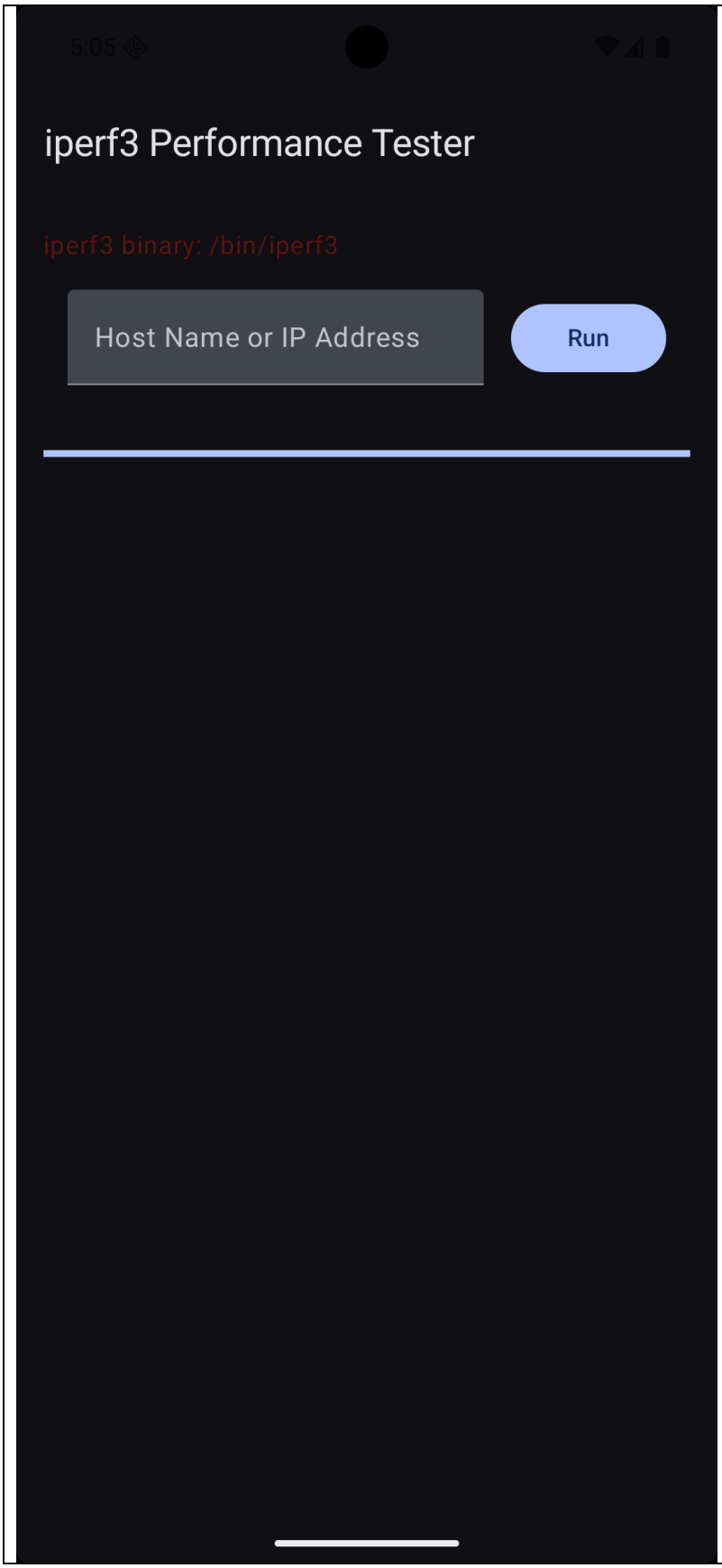

5:08
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
Run
Tested for 10 second(s)
Return Code: 1
iperf3: error - unable to connect to server: No address associated with hostname


5:08
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
Run
Tested for 10 second(s)
Return Code: 1
iperf3: error - unable to connect to server:
Connection timed out


5:07
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
Run
Tested for 10 second(s)
Return Code: 0
Average: 367.0 Mbits/sec
Maximum: 398.0 Mbits/sec
Minimum: 314.0 Mbits/sec
Connecting to host jabramson.com, port 5201
Reverse mode, remote host jabramson.com is sending
Local Host/IP: 10.0.2.16
Remote Host/IP: 192.168.1.32
Remote Port: 5201
Running 0.00-1.00 314 Mbits/sec
Running 1.00-2.00 376 Mbits/sec
Running 2.00-3.00 380 Mbits/sec
Running 3.00-4.00 317 Mbits/sec
Running 4.00-5.00 360 Mbits/sec
Running 5.00-6.00 381 Mbits/sec
Running 6.00-7.00 382 Mbits/sec
Running 7.00-8.00 365 Mbits/sec
Running 8.00-9.00 387 Mbits/sec
Running 9.00-10.00 398 Mbits/sec
Results
0.00-10.00 sender 367 Mbits/sec

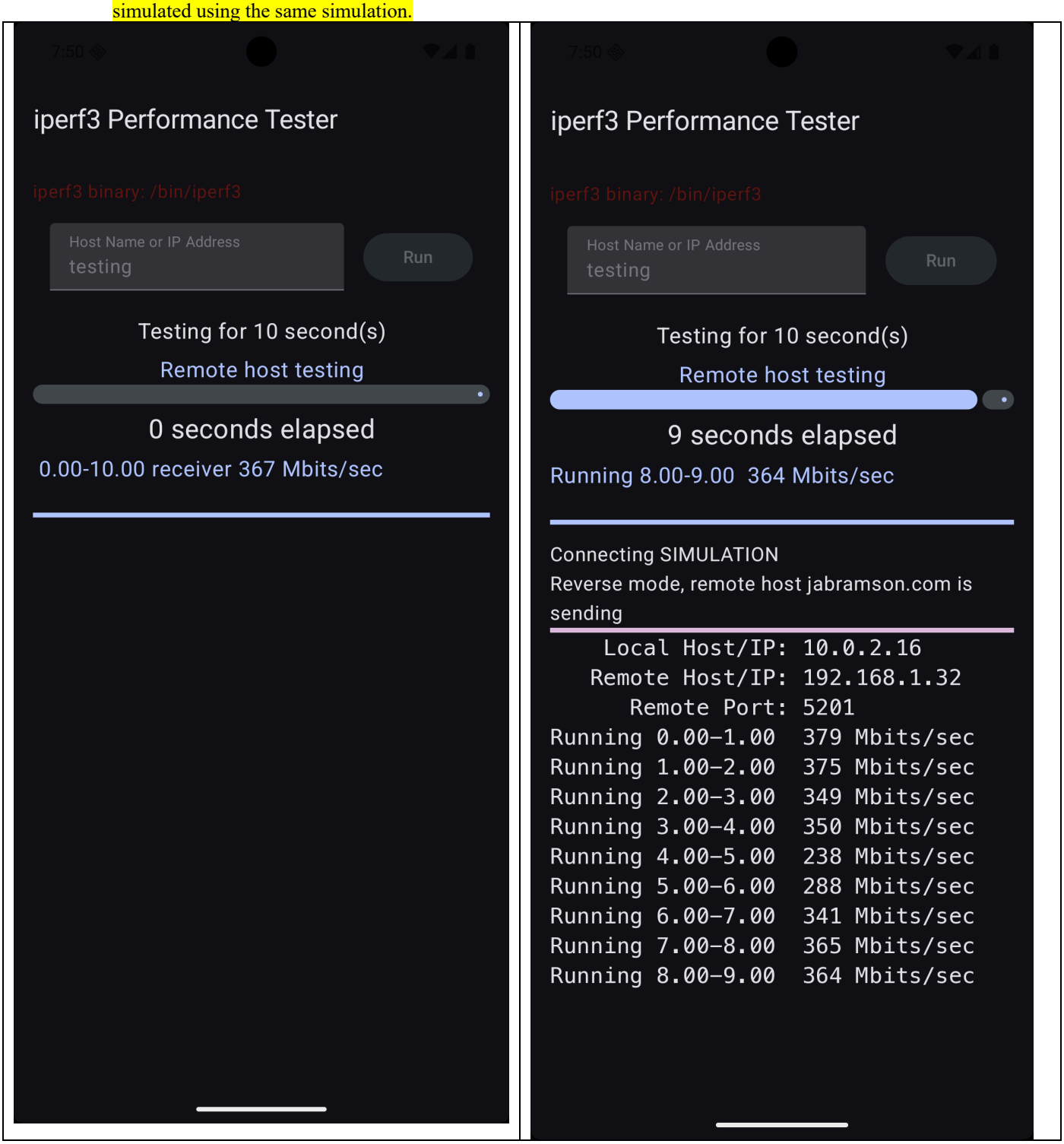

5:07
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
jabramson.com
Run
Testing for 10 second(s)
Remote host jabramson.com
2 seconds elapsed
Running 1.00-2.00 376 Mbits/sec
Connecting to host jabramson.com, port 5201
Reverse mode, remote host jabramson.com is sending
Local Host/IP: 10.0.2.16
Remote Host/IP: 192.168.1.32
Remote Port: 5201
Running 0.00-1.00 314 Mbits/sec
Running 1.00-2.00 376 Mbits/sec


5:07
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
jabramson.com
Run


5:05
iperf3 Performance Tester
iperf3 binary: /bin/iperf3
Host Name or IP Address
Run

7.1. Fallback simulation

- As a last-ditch fallback, if you use the hostname `testing` then the iperf3 network traffic is simulated using the same simulation.



AI Usage Log

Tool	Task	Evaluation	Links or prompt history
1	https://claude.ai	Positive	Claude AI Public Link

TBD

- ## Future Work (Optional)

  Rework application to utilize NDK and JNI bindings.

  Deploy to Android play store and begin marketing/sales

- ## Project Demo Links

  (*For on campus students, we will have project presentations in class. For online students, you are required to submit a video of your project presentation which includes a demo of your app and explanation of your implementation. You can use Kaltura or zoom or any video tool to make the video and then submit it on blackboard. Please check the following link for the details of using Kaltura to make and submit videos on blackboard. You can also use other video tools and upload your video to youtube if you like: https://onlinecampus.bu.edu/bbcswebdav/courses/00cwr_odeelements/metcs/cs_Kaltura.htm*)

- ## References

  (*any references you used for the project*)